EC 504
Fall, 2025
Homework 7

Due Tuesday, December 9, 2025, 11:59 pm

Problem 7.1 (10 pts)

For the KMP algorithm, compute the prefix function for the following string: "abracadabra".

Solution:

$\pi = [0, 0, 0, 1, 0, 1, 0, 1, 2, 3, 4]$

Problem 7.2 (15 pts)

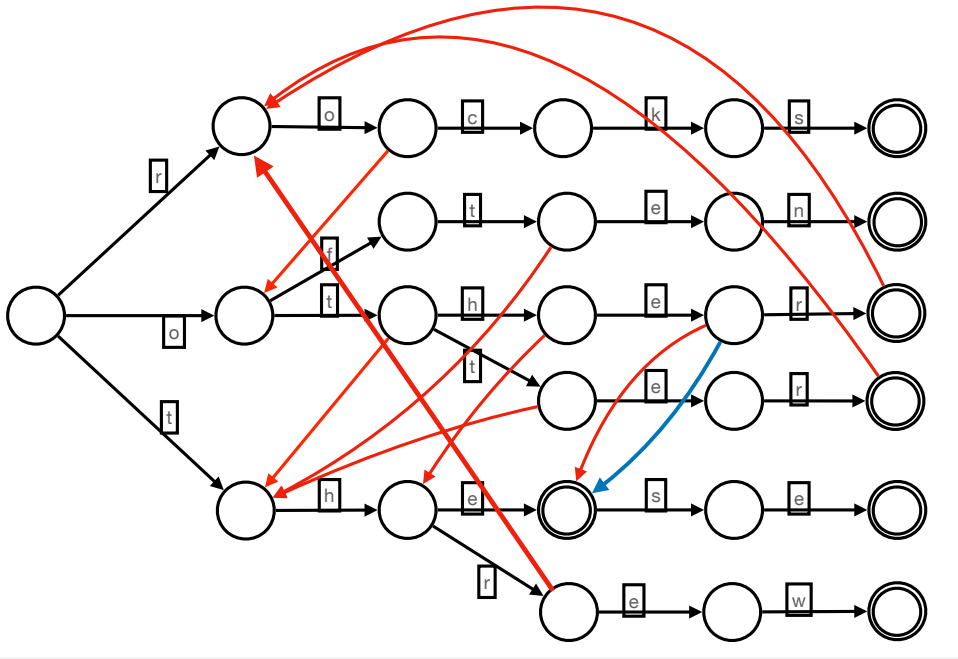
For the Aho-Corasick algorithm, compute the suffix trie that would be used to search for the following strings simultaneously:

```
"other"
"otter"
"often"
"threw"
"these"
```

You do not need to include output links, only prefix links.

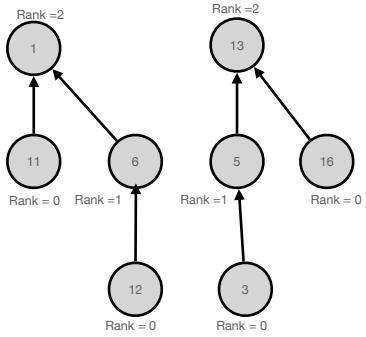
Solution:

See figure below. Output links were not needed in the solution (the blue link), but are included here for completeness.

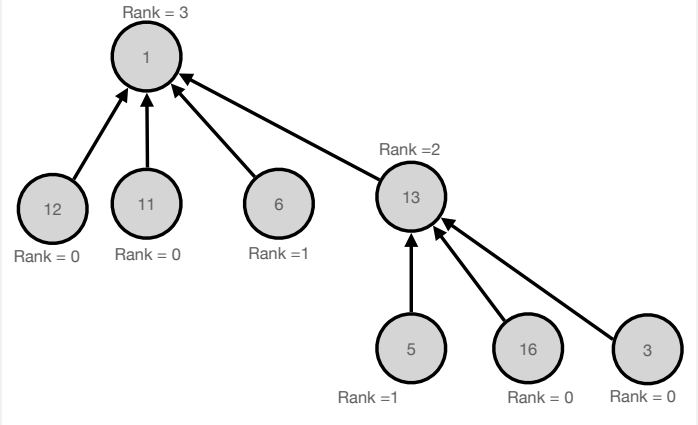


Problem 7.3 (5 pts)

Old exam question: Consider the two trees, shown on the right, that are part of a disjoint set forest. Suppose we find a relation between keys 12 and 3. Show the disjoint set tree that results from the union operation using merge by rank and path compression: if two roots have the same rank, merge the larger value root under the smaller value root and upon find, compress the paths.



Solution:



Problem 7.4 (15 pts)

Assume you have a scheduling problem with 10 customers. Customer i has value V_i, and requires processing time T_i. The values of V_i and T_i are listed below as arrays:

```
V = {3,4,7,5,2,3,5,8,9,6}
T = {2,3,4,3,1,3,3,5,6,4}
```

Assume that jobs can be scheduled at any start time, and there is a total amount of time T to allocate for the jobs. Hence, you can view the scheduling problem as a version of a knapsack problem.

Suppose that jobs must be scheduled fully to receive any value, so that no partial credit is given to incomplete jobs. Use dynamic programming to find the optimal value which can be scheduled in a total of 9, 10, 11, 12 time units. (Note: this may be easier to do in MATLAB or Python than by hand...)

Solution:

This is an integer knapsack problem, which we solve with dynamic programming. The optimal value function is a function $J(i, C)$, where i is the largest integer for the tasks that are being considered and C is the available capacity. We use the relationship:

$$J(i, C) = min\{J(i-1, C), V_i + J(i-1, C-T_i)\}$$

The table is shown below:

\ c = 0 i:	1	2	3	4	5	6	7	8	9	10	11	12
1	0	0	3	3	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7	7	7
3	0	0	3	4	7	7	10	11	11	14	14	14
4	0	0	3	5	7	8	10	12	12	15	16	16
5	0	2	3	5	7	9	10	12	14	15	17	18
6	0	2	3	5	7	9	10	12	14	15	17	18
7	0	2	3	5	7	9	10	12	14	15	17	19
8	0	2	3	5	7	9	10	12	14	15	17	19
9	0	2	3	5	7	9	10	12	14	15	17	19
10	0	2	3	5	7	9	10	12	14	15	17	19

From the table, the optimal values for capacities 9, 10, 11, 12 are 15, 17, 19 and 20.

The optimal assignments are: For capacity 9, the tasks scheduled are: 1, 3, 4.

For capacity 10: the tasks scheduled are: 1,3, 4, 5.

For capacity 11: the tasks scheduled are: 3, 4, 5, 7.

For capacity 12: the tasks scheduled are: 1,3,4,7.

Problem 7.5 (15 pts)

Consider the two words: *perplex* and *peruse*. Consider them as a sequence of letters. We would like to find the best alignment of the two letter sequences, with the following costs:

- Incur a penalty of 1 for any skipped letter.
- Incur a penalty of 3 for matching a consonant to a vowel.

- Incur a penalty of 2 for mismatching a consonant to another consonant or a vowel to another vowel.
- Incur a reward of 1 for correctly matching two letters.

Using this reward structure, find the minimum cost alignment. State the cost of the alignment.

Solution:

Minimum cost = 1. Best alignment is shown below.

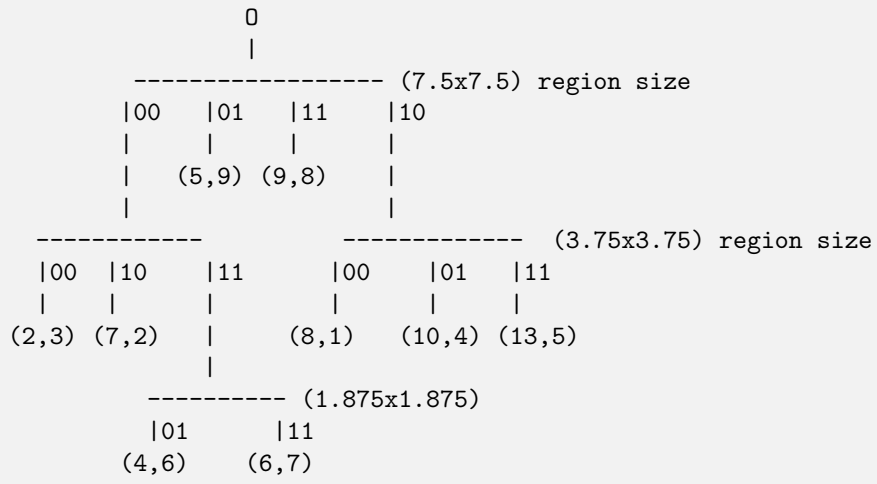
		p	e	r	p	l	e	x
	0	1	2	3	4	5	6	7
p	1	-1	0	1	2	3	4	5
e	2	0	-2	-1	0	1	2	3
r	3	1	-1	-3	-2	-1	0	1
u	4	2	0	-2	-1	0	1	2
s	5	3	1	-1	0	1	2	3
e	6	4	2	0	1	2	0	1

p	e	r	p	l			e	x
p	e	r			u	s	e	

Problem 7.6 (15 points) Consider the following pairs of coordinates: (7,2), (2,3), (4,6), (5,9), (8,1), (10,4), (13,5) (6,7), (9,8). Arrange them in a quadtree, using the maximum range 0-15 for each of the coordinates.

Solution:

The boundaries for the quadtree areas will be $15/2^k$ for some integers k. Since the boundaries will not be integers, we won't have any points on the boundaries. The binary codes 00, 01, 10, 11 indicate which branch it is; 00 is the smallest x, smallest y (Southwest), 01 is smallest x, largest y (Northwest), 10 is largest x, smallest y (Southeast), and 11 is largest x and y (Northeast).

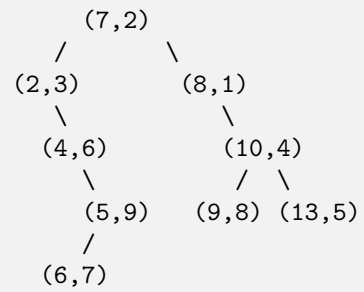


Problem 7.7 (10 points)

Consider the following pairs of coordinates: (7,2), (2,3), (4,6), (5,9), (8,1), (10,4), (13,5) (6,7), (9,8). Show the 2-d binary search tree which results from inserting these elements in the order given, with no attempt to balance them.

Solution:

Remember we alternate coordinates in the insertion comparisons. The root node is obvious.



Problem 7.8 (15 points)

Consider the following pairs of coordinates: (7,2), (2,3), (4,6), (5,9), (8,1), (10,4), (13,5) (6,7), (9,8). Form a balanced 2-d binary search tree using the median of the elements to split. To make things unique, when you have to find a median of a list with an even number of entries, choose the smaller of the two entries as the median.

Solution:

Start by finding the median value of the first coordinate of all the points. Sorting by first coordinate yields the list:

$$(2,3),(4,6),(5,9),(6,7),(7,2),(8,1),(9,8),(10,4),(13,5).$$

The median value is 7, so we use 7 as the root key for the first split.

For the left subtree, the navigation key will be the median of the y coordinates of the entries that branch left. There are five entries: (2,3), (4,6), (5,9), (6,7), so the median y coordinate is 6. For the right subtree, there are four entries: (8,1), (9,8), (10,4), (13,5), so there are two choices of median, so per the problem instructions, we pick the smaller, which is 4.

We repeat this process, alternating between branching on an x coordinate or a y coordinate